

Exercise - Snake Game with Turtle

Python OOP, Turtle GUI, Events and File Handling

In this exercise, you will develop the classic Snake game using Python and the Turtle module. The game must be built from scratch and should demonstrate object-oriented programming, keyboard listeners, score management, game loop control and persistent file storage.

Exercise Overview

- **Goal:** Build a fully working Snake game using Python Turtle.
- **Interface:** Desktop-based Turtle window. No CLI input should be used during the game.
- **Development approach:** Students must develop the solution from scratch - **no AI involvement**.
- **Demonstration:** An attached photo is provided as a visual demonstration of the expected game appearance.

General Guidelines

- This exercise must be solved individually.
- The game must be implemented using the Turtle module.
- The solution must be written clearly, with meaningful names for classes, methods and variables.
- Do not print to the console.
- The program must run without runtime errors or warnings.
- The game must be playable using keyboard listeners only.

Required Architecture

The solution must use object-oriented programming. At minimum, implement the following classes:

Class	Responsibility	Examples of Methods
Snake	Create, move, grow and control the snake direction.	move(), grow(), up(), down(), left(), right()
Food	Create food and relocate it after being eaten.	refresh(), random_position()
Scoreboard	Display current score, highest score and game over message.	update_score(), game_over(), read_highest_score(), save_highest_score()

Game Setup Requirements

- Screen size: 600 x 600 pixels.
- The playable border limit is ± 288 on both the x-axis and y-axis.
- The snake must start with 3 segments.
- Each snake segment is Turtle-sized: 20 x 20 pixels.
- Each movement step is 20 pixels.
- The snake starts moving to the right automatically when the program starts.
- The player cannot move the snake directly. The snake keeps moving by itself and the user can only change its direction.

Keyboard Controls and Listeners

Key	Action
Up Arrow	Change direction to up

Down Arrow	Change direction to down
Left Arrow	Change direction to left
Right Arrow	Change direction to right

- Use Turtle keyboard listeners to control direction changes.
- Reverse movement is not allowed. For example, if the snake is moving right, it cannot immediately move left.

Food Requirements

- The food must be displayed as a circle.
- Each time the snake eats food, the food moves to a different random location inside the board.
- Food cannot appear on any snake segment.
- Use collision detection such as `turtle.distance(food) < 15` to determine whether the snake has eaten the food.

Score and Highest Score Requirements

- At the top center of the board display both scores "Your Score" and "Highest score".
- Each time the snake eats food, the current score increases by 1.
- After every food collision, update both the current score and highest score display if needed.
- If the current score becomes greater than the highest score during the game, the highest score update live on the screen.
- The highest score will be loaded from a text file named "highest_score.txt".
- If highest_score.txt does not exist, create it automatically and write 0 into it.
- If highest_score.txt exists but is empty or invalid, treat the highest score as 0.
- On game failure only, save the current score to "highest_score.txt" if it is greater than the stored highest score.
- If the user manually closes the window, no special save action is required.

Failure Conditions

- The game fails if the snake collides with the screen border.
- Border collision should be detected when the snake head crosses $x > 288$, $x < -288$, $y > 288$, or $y < -288$.
- The game fails if the snake head collides with any part of its own tail.
- You may use distance-based collision detection, for example `turtle.distance(segment) < 15`.
- When the game fails, display a visible "GAME OVER" message on the screen.

Game Speed Requirements

- The game loop should begin with a sleep delay of 0.1 seconds.
- Each time the snake eats food, the game speed should increase by decreasing the sleep delay by 0.01 (Ex. 0.1 to 0.09).
- The minimum sleep delay is 0.05 seconds. The game should not become faster than this limit.
- Use `time.sleep(delay)` inside the main game loop.
- Use `screen.update()` inside the main game loop to refresh the screen manually.

Useful Functions and Hints

Function / Concept	Purpose
<code>turtle.distance(food) < 15</code>	Detects collision between the snake head and the food.
<code>turtle.distance(segment) < 15</code>	Detects collision between the snake head and its tail.
<code>screen.listen()</code>	Allow the Turtle screen to start listening to keyboard events.
<code>screen.exitonclick()</code>	Keep the Turtle window open until the user clicks to close it.
<code>screen.update()</code>	Refresh the screen manually when using manual screen updates.

time.sleep(delay)

Control the movement speed of the snake.

Expected Program Flow

1. Initialize the screen with size 600 x 600.
2. Create the snake with 3 segments and position that, as follows:
 - Head (first turtle) = (0,0)
 - Second turtle = (-20, 0)
 - Third turtle = (-40, 0)
3. Create the food object.
4. Create the scoreboard and load the highest score from highest_score.txt.
5. Register keyboard listeners for direction changes.
6. Start the main game loop.
7. Move the snake automatically every iteration.
8. Check if the snake ate food.
9. If food was eaten, increase score, grow snake, move food, update scoreboard and increase speed.
10. Check border collision and tail collision.
11. If failure occurs, update the highest score file if needed and show GAME OVER.

Submission Requirements

- TBD

Grading Criteria

Criteria	Focus
Correct game logic (45 pt)	Snake movement, food eating, growth, collision detection
OOP design (25 pt)	Correct use of classes, attributes and methods
Documentation (5)	When needed additional information about the logic.
Code quality and efficiency (25)	Readable, modular and without unnecessary output

Bonus (Extra Credit of 20 pt)

- Add a start screen before the game begins.
- Add a restart option after GAME OVER.
- Add different food colors or random food values.
- Add a sound effect if your environment supports it.
- Add levels or visual border styling.

Reminder: The attached video is for demonstration only. The implementation must be your own work.